Q-App Package

Describes how to create a Q-App; what is included in the Q-App file and the elements comprising its JSON configuration file.

Table of Contents

- Overview
- File structure
- Q-App creation best practices
- config.json file
- "storage" content action type
- "import" content action type
- "dbScript" content action type
- "manual" content action type
- json.config example
- Zip & go
- See also

Overview

Q-Flow Apps, or "Q-Apps" is a feature that extends Q-Flow functionality through custom apps added on top of the Q-Flow solution. Using this feature you consolidate customization projects—which are a combination of configuration settings, scripts, UI templates and more—into Q-Apps. The Q-App is a package that includes a JSON configuration file and any other file(s) comprising the app.

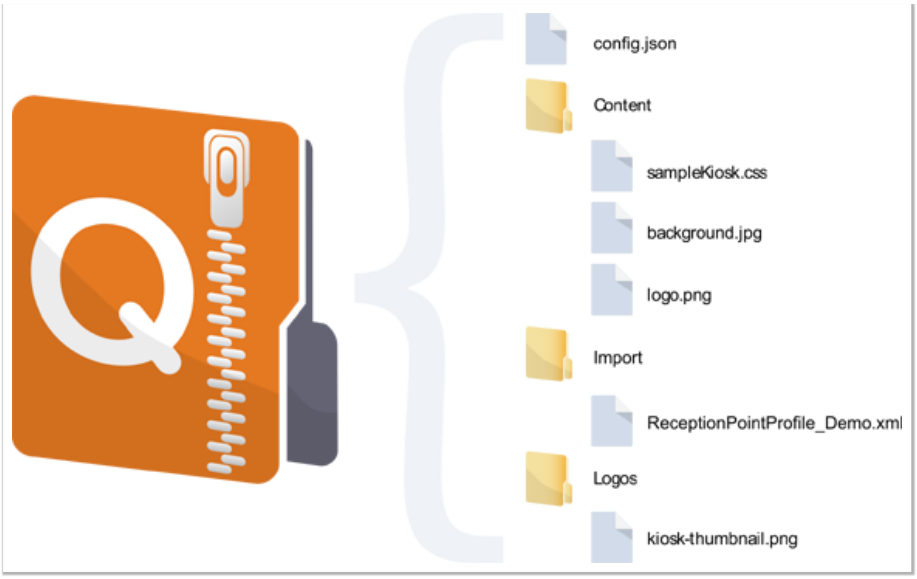
Q-Apps by definition are portable; they can be installed on a new server, transferred between different environments (dev, test, pre-production, production etc.), and shared with others through the Q-nomy app market ("Q-Market").

Q-App versions are managed by Q-Flow; administrators can easily tell which version of a Q-App they are using and if necessary roll back to the previous version. The use of Q-Apps in itself does not require any particular license. However, if a Q-App applies to a particular module, licensing is required to use that module as part of the app — same as in any Q-Flow implementation.

Technically, a Q-App is a single ZIP file whose extension is changed from *.zip* to *.qapp*

File structure

A Q-App is constructed from a `config.json` configuration file at the root level, and the folders and files to which the configuration file points to.
The Q-App's folder is compressed into a ZIP file with the Q-App's name and version, and the "zip" extension is changed to the "qapp" proprietary extension. For example:



Attached are two versions (an initial version and a later version) of a Q-App for the same kiosk reception point. Download and experiment with them for a hands-on learning experience:

Sample Q-App version 1
Sample Q-App version 2

Q-App creation best practices

- To prevent conflicts with other apps, avoid using generic fields such as "External Reference" or "Description" for saving data used for customization. When possible, use other methods such as content parameters and custom properties.
- When running a DB script as part of the app installation, provide a rollback script as well.
- Split the app into the maximum number of components to facilitate their tracking and management.
  For example, it is not recommended using a single DB script to create multiple objects.

config.json file

Each Q-App must include a configuration file with the name “config.json”. The configuration file includes a declaration of the app’s content and properties as described below with supporting examples.

id

App ID is a unique identifier for the Q-App. All versions of an app use the same GUID generated for the app's initial version, and each Q-App has a unique value. As each Q-App requires a unique identifier it is the responsibility of the developer to generate a new GUID for each Q-App.

```
"id": "A831175A-AEE1-49EE-8D2D-BCB82B9C1907"
```

> **Note**
> Do not tweak an existing GUID manually to create additional GUIDs as doing so runs a high risk for duplicate identifiers.

versionId

Version ID is a unique identifier of the version. This identifier must be updated for each version of the Q-App.

```
"versionId": "461b44c6-591e-432e-b738-ab0564edc59e"
```

> **Note**
> Do not tweak an existing GUID manually to create additional GUIDs as doing so runs a high risk for duplicate identifiers.

name

Name is the name given to the app; this name is displayed in Q-Flow, so make sure the name is distinct to make the Q-App easy to recognize.
Q-Apps may be shared and transferred, therefore avoid using generic names or specific customer names.

```
"name": "app name"
```

version

Version is the version number of the app. The version should follow the versioning convention of [major].[minor].[build]. This is very important as Q-Apps may be shared and transferred.

- Major: Increment when there are significant changes
- Minor: Increment when there are minor changes
- Build: Increment for internal tracking purposes.

> **Note**
> The version numbers of later versions or updates should be higher than the initial version in ascending order.

```
"version": "1.0.0"
```

logo

The path to the app's logo file, i.e. the logo that identifies the Q-App. The logo file should be placed in a folder that will be included in the package.
If a logo is not supplied, Q-Flow will make use of a default image.

```
"logo": "LogoFolder/logo.png"
```

The logo file is displayed in the Q-App's card on the My Q-Apps page.
Recommended logo dimensions: 80 pixels x 80 pixels. The supported file types are per the extensions set in the Images Allowed Extensions System Settings parameter.

description

A short description of the app. The description should be informative and describe what the Q-App is for, and what it can do.

```
"description": "short description of the app"
```

developer

The name of the developer or organization that created the Q-App.

```
"developer": "developer name"
```

qflowVersions

The list of Q-Flow versions this Q-App supports. Ending a value with "+" indicates that all versions later than the specified version are supported (including future versions).

```
"qflowVersions": ["6.0", "6.1+"]
```

If the Q-App makes use of a Q-Flow feature that is not available in all versions, you need to indicate this in the `qflowVersions` information. For example, you would list "6.1+" to prevent installing this Q-App on version 6.0.

> **Note**
> Only major and minor version numbers should be used. There is no support for version build number and service-pack information.

license

The list of license modules required to run this package.
For example:

```
"license": [ { "module": "Herald"}, { "module": "Connect" }  ]
```

The list is validated before running the Q-App to ensure the Q-App can be used.

Following is the list of modules and their corresponding configuration values.

Module	Config Value
Advanced Calendar	AdvancedCalendar
Advanced Queue Management	AdvancedQueueManagement
Advanced UXC	AdvancedUXC
Q-Flow API	API
BPM	AutoFlow
Brochure POD	Brochure
Brochure POD API	BrochureAPI
Calendar	Calendar
Connect	Connect
Exchange Bridge	ExchangeBridge
Herald	Herald
Intercom	Intercom
Online Form	OnlineForm
Planner API	PlannerAPI
Queue & Route	QueueManagement
Staff Optimization	Staffing
User Experience Customization	UserExperienceCustomization
Windows Printing	WindowsPrinting

requiredPreviousVersion

Indicates the version of the Q-App that should already be installed for the Q-App update to install successfully. This is a prerequisite when installing a Q-App update that provides only changes, and not a complete app.

```
"requiredPreviousVersion": "1.0.0"
```

defaultUpgradeAction

"default upgrade action" determines how to proceed with import and storage actions in case the item already exists (was installed in a previous version). Supported values are:

- “override” - Items included in the new version will overwrite existing items from previous versions.
- “leave” - Existing items from previous versions will be kept, and new items from the Q-App being installed will be ignored.

If no value was provided, or an invalid value was provided, the default value is “override”.

The setting here provides a default upgrade action for all the Q-App items; but an item's specified upgrade action setting will override this general setting.

```
"defaultUpgradeAction": "override"
```

defaultRemovedItemsAction

"default removed items action" provides the developer with control over what is done with items that exist in the version that is currently installed, but do not exist in the new version that is being installed.
Supported values are:

- “remove” - Existing items that are not included in the new version will be removed. This option is usually used when installing a complete Q-App update that provides a complete app and not just updates.
- “leave” - Existing items that are not included in the new version will be kept. This option usually goes with the `requiredPreviousVersion` attribute, which enables developers to create upgrade versions that include only updates.

If no value or an invalid value was provided, the default value is “leave”.

```
"defaultRemovedItemsAction": "leave"
```

defaultOverrideAction

The default override action parameter applies only to content templates, classifications and appointment types. This parameter determines how the import action should proceed in case an item with the same name already exists. Supported values are:

- “override” - Properties of the existing objects are replaced with the values from the Q-App being installed.
- “leave” - leaves the existing object and ignores the item from the Q-App being installed.
- "exception" - throws an exception and terminates the import action (changes made by the Q-App being installed are rolled back).

If no value or an invalid value was provided, the default value is “exception”.

```
"defaultOverrideAction": "exception"
```

Parameters

The parameters node is an array that specifies the parameters that the user installing the app will be required to supply values for.
After the parameter's values are supplied, the developer can use these parameters and supplied values in the DB script or Import actions.
See How to use Q-App Parameters for more information.
For example:

```
"parameters":
  [{
        "name": "@Parameter1",
         "type" : "ServiceFilter",
         "required" : "false",
         "description" :"select the service filter that will be used to filter services in the kio
    },
    {
        "name": "@ServiceName",
        "type" : "text",
        "required" : "true",
        "description" :"Enter the service name to be mapped to the kiosk"
  }]
```

Properties are as follows:

name

The parameter's name. Must start with @. The name should be formed using alphabetic characters, numbers, and no spaces.

type

The following values are supported:

- AppointmentType
- Assistant
- AvatarTheme
- BackOfficeTask
- BrochureProfile
- CalendarTemplate
- Classification
- ClassificationGroup
- CustomerLevel
- CustomMenu
- CustomPage
- EntityType
- Group
- HolidayGroup
- InfoPageProfile
- IntercomMessageTemplate
- MonitorType
- Procedure
- ReceptionPointProfile
- Report
- Resource
- ResourceType
- Schedule
- Script

  The "Script" type property requires another property "scriptType" whose supported values are:

- BrochureProfile
- CompositeAppointment
- CustomPage
- Group
- InfoPageProfile
- NotificationChannel
- Procedure
- ReceptionPointProfile
- Resource
- Service
- System

- ServiceAttribute
- Service
- ServiceFilter
- ServiceProfile
- ServiceType
- SignOutReason
- Skill
- SkillType
- SlotReservationReason
- SoftKey
- Ticket
- Unit
- UnitType

The following two special values are supported as well:

- ID - for manual mapping of IDs
- Text - for manual text input from users installing the Q-App

required

Defines whether the user is required to map this parameter or not. The default value is “true”.

> **Note**
>
> When the value of the "type" property is set to "text", the value of this property ("required"), must be "true".

description

The description should provide useful information about the parameter.

dependencies

The dependencies node is an array that declares dependencies. When installing a Q-App with declared dependencies, the installed Q-App checks whether the Q-Apps this Q-App is dependent upon are installed. The installation does not continue until they are. For example, the OR Planner Q-App has a Spanish language pack; declaring a dependency to the language pack ensures the language pack will not be installed until OR Planner is installed. Thus new projects can consider generic solutions available on Q-Market as a given, and make use of the functionality those Q-Apps provide in the solutions they are creating.

Properties

- `qappId` - the ID of the target Q-App, i.e. the Q-App to which there is a dependency.

- `qappName` - the name of the target Q-App.
- `minimumVersion` (optional) - the minimum version of the target Q-App.
- `downloadUrl` (optional) - the URL that leads to the page from which the target Q-App can be downloaded. Usually this URL will point to the target Q-App's Q-Market URL.

For example:

```
"dependencies": [
    {
      "qappId": "659ab398-c178-4f5c-a5fb-3972b7119ee9",
      "qappName": "Report Center",
      "minimumVersion": "1.0.0",
      "downloadUrl": "https://qmarket.qnomy.com/market/details/659ab398-c178-4f5c-a5fb-3972b7119ee
    },
  {
      "qappId": "84d9541a-5163-46e0-b3df-4c4485301400",
      "minimumVersion": "",
      "qappName": "Basic Kiosk for Checking in",
      "downloadUrl": ""
    }
  ]
```

Content

The content node is an array that lists the app's content, built in the following format:

```
"content": [
    {
      "storage": {
        "name": "Storage Files",
        "type": "folder",
        "path": "images",
        "target": "images",
        "rollbackAction": "remove",
        "upgradeAction": "override"
      }
    },
    {
      "storage": {
        "name": "Background",
        "type": "file",
        "path": "images/1.png",
        "target": "images",
        "rollbackAction": "remove",
        "upgradeAction": "override"
      }
    },
    {
      "import": {
        "name": "Kiosk1",
        "type": "ReceptionPointProfile",
        "path": "file1.xml",
        "createParameter" : "@Parameter2",
        "upgradeAction" : "leave",
        "overrideAction" : "override"
      }
    },
    {
      "dbScript": {
        "name": "Create Job 1",
        "type": "job",
        "path": "Scripts/job.sql",
        "target": "qflow",
        "rollbackPath": "Scripts/rollbackJob.sql",
```

```
      "manualScript": "True"
    }
  },
  {
    "manual": {
      "name": "myApp_serviceScript DLL",
      "path": "myApp_serviceScript.dll",
      "instructions": "Download and copy the file to the BIN folder on every Q-Flow web server",
      "rollbackInstructions": "Remove the file 'myApp_serviceScript.dll' from the BIN folder"
    }
  }
]
```

Each item the in the array may be one of the following types, which determines how to handle the item when the app is installed:

- storage
- import
- dbScript
- manual

Each type is elaborated in the following sections.

"storage" content action type

A content action that copies the specified file or folder to the specified target folder in Q-Flow Storage.

Properties

- `name` - item names need to follow these rules:
  1. Each item's name should be unique to distinguish items one from the other.
  2. An item's name should be the same name it was named in previous versions. If you do not keep the item's name, the item will not be identified correctly resulting in a brand new item instead of creating a new version for the existing item.
- `type` - one of the following values:
  - "folder" - copy all files from the folder specified in the property `path` to the target container in Q-Flow storage. If needed, see Creating Folder Hierarchies in Q-Apps.

    > **Note**
    > Only single-level folders can be uploaded to Q-Flow Storage. Folders containing subfolders will trigger an exception.

  - "file" - copy the single file specified in the property `path` to the target container in Q-Flow storage.
- `path` - a path to the file or folder to be used. The path is relative to the root of the qapp file.
- `target` - the name of the target container in Q-Flow storage.
- `rollbackAction` - one of the following values:
  - "remove" - removes files from storage.
  - "leave" - leaves the files in storage.
- `upgradeAction` - one of the following values:
  - "override" - in case the file already exists in storage, overwrite the existing file with the file provided in the package.

- "leave" - in case the file already exists in storage, leave the existing file and do not use the file provided in the package.

"import" content action type

A content action that imports Q-Flow objects from the Q-App to Q-Flow.

Properties

- `name` - item names need to follow these rules:
  1. Each item's name should be unique to distinguish items one from the other.
  2. An item's name should be the same name it was named in previous versions. If you do not keep the item's name, the item will not be identified correctly resulting in a brand new item instead of creating a new version for the existing item.
- `type` - the type of object to import. See tables below. The [type] property supports any of the following values. The values are divided to two groups, which differ in their rollback and upgrade behavior.

  The first group includes objects with structured history. Structured history is a mechanism that saves the entire object in the Q-Flow database prior to upgrading it to a new version. These objects can be rolled back to their last saved instance.
  Objects belonging to this group are listed in the following table:

Object Name	Config Value	Comments
Script	Script	
Brochure Profile	BrochureProfile	Brochure page cannot be imported separately
Info Page Profile	InfoPageProfile	Info Page Playlists cannot be imported separately
Reception Point Profile	ReceptionPointProfile	Reception Point page cannot be imported separately
Custom Page	CustomPage	Custom Form cannot be imported separately
Workflow Procedures	Procedure	
Online Form Template	OnlineFormTemplate	
Assistant	Assistant	
Calendar Template	CalendarTemplate	
Monitor Commands	MonitorCommand	Single Monitor command in each action.
Notification Template	NotificationTemplate	
Alert Rules	AlertRule	Alert rules imported without Recipients (users and groups)
Schedule	Schedule	
Service Filter	ServiceFilter	
Ticket	Ticket	
Custom JavaScript	CustomJavaScript	
Avatar Theme	AvatarTheme	
Holiday Group	HolidayGroup	

Service Profile	ServiceProfile	
Skill Type	SkillType	
Group	Group	
Resource Type	ResourceType	
Webhook[v6.2]	Webhook	
Application Event[v6.2]	ApplicationEvent	
Soft Key[v6.2]	SoftKey	

The second group includes objects that are not saved in the Q-Flow database as these objects are subject to change from various sources. When a Q-App is installed these objects are overwritten with their corresponding objects within the Q-App. Rolling back an installed Q-App will leave these items unchanged.
Objects belonging to this group are listed in the following table:

Object Name	Config Value	Comments
Content Template + Category	ContentTemplate	
Classifications + Classification Groups	Classification	
Service Type + Appointment Types	ServiceType	
Connect Channel Provider[v6.2]	NotificationProvider	
Custom Role (and tasks)[v6.2]	CustomRole	
Entity Type[v6.2]	EntityType	
QBP (except built-in QBPs)[v6.2]	QBPTask	
Custom API[v6.3]	CustomApi	Will be supported from Q-Flow 6.3

- `path` - a path to the file or folder to be used. The path is relative to the root of the zip file.
- `createParameter` - the name of the parameter to be automatically created by Q-Flow after importing this object. The value of the parameter is the ID of the newly created object.
- `upgradeAction` - determines how to proceed with the import action in case the item already exists (was installed in a previous version).
  Supported values are:
  - “override” - Items included in the new version will overwrite existing items from previous versions.
  - “leave” - Existing items from previous versions will be kept, and new items from the Q-App being installed will be ignored.

  If no value was provided, or an invalid value was provided, the DefaultOverrideAction value is used.
- `overrideAction` - The override action parameter applies only to content templates, classifications and appointment types. This parameter determines how the import action should proceed in case an item with the same name already exists.
  Supported values are:
  - “override” - Properties of the existing objects are replaced with the values from the Q-App being installed.

- “leave” - leaves the existing object and ignores the item from the Q-App being installed.
- "exception" - throws an exception and terminates the import action (changes made by the Q-App being installed are rolled back).

If no value or an invalid value was provided, the default value is the value set for the DefaultOverrideAction.

"dbScript" content action type

A content action that runs the specified file as an SQL script on Q-Flow database. All the known parameters are sent to the database with the script.

> **Note**
>
> SQL Management Studio's "GO" command is not supported in the Q-App installation. (If used in a Q-App database script, an exception will be issued when the Q-App is installed.)

The script writer can return any number of parameters from the script, by returning a result set with these columns – `ParameterName`, `ParameterValue`, `ParameterType` (Id = 0 ,Text = 1).
Every returned new parameter is added to the mapped parameters.

Properties

- `name` - item names need to follow these rules:
  1. Each item's name should be unique to distinguish items one from the other.
  2. An item's name should be the same name it was named in previous versions. If you do not keep the item's name, the item will not be identified correctly resulting in a brand new item instead of creating a new version for the existing item.
- `type` - supports the following:
  - "storedProcedure" – no parameters
  - "table" - no parameters
  - "Data" - with parameters
  - "Job" - no parameters
  - "Script" - with parameters
- `path` - a path to the file or folder to be used. The path is relative to the root of the zip file.
- `target` - supports the following values:
  - {empty string} or "qflow" – run the script on Q-Flow database
  - "attachment" – run the script on the attachment database
  - "dispatcher" – run the script on the dispatcher database
- `rollbackPath` - a path to a script file to be executed when a rollback is done on the version.
- `manualScript` - determines whether the script can be executed as a standalone script.
  Default value “false”.
  Developers should include validation and checks for existence of the object in the creation script.
  Each dbScript item can have a specific script in the configuration file to be executed in rollback.

  A parameter called `@Parameters AS XML` is available when executing scripts or data.
  The parameter includes all the mapped and created parameters.

```
DECLARE @Parameters XML = "<Parameters><Parameter Name="@Example" Value="abc123" /></Paramete
```

For example:

```
SELECT @Name = T.c.value('@Value','NVARCHAR(50)')
FROM @Parameters.nodes('/Parameters/Parameter') T(c)
```

"manual" content action type

A content action that provides the user with files and/or instructions to be completed manually. For example, provide the user a DLL file to copy to the bin folder of the Q-Flow installation.

Properties

- `name` - item names need to follow these rules:
  1. Each item's name should be unique to distinguish items one from the other.
  2. An item's name should be the same name it was named in previous versions. If you do not keep the item's name, the item will not be identified correctly resulting in a brand new item instead of creating a new version for the existing item.
- `path` - a path to a file that the user may download. The path is relative to the root of the zip file.
- `instructions` - detailed instruction for the user on how to complete a upgrade or install.
- `rollbackInstructions` - detailed instruction for the user on how to complete the rollback.

json.config example

```
{
  "id": "AAB27274-D622-4002-9604-96CD623E9ECD",
  "versionId": "8559BFAA-5FC2-4370-8728-DC659CA551FE",
  "name": "Kiosk Demo Q-App v1",
  "version": "1.0.0",
  "logo": "Logos/kiosk-thumbnail.png",
  "description": "Kiosk Demo Q-App",
  "developer": "Q-nomy Dev Team",
  "qflowVersions": [ "6.0+" ],
  "license": [
    { "module": "AdvancedQueueManagement" },
    { "module": "UserExperienceCustomization" }
  ],
  "defaultRemovedItemsAction": "leave",
  "defaultUpgradeAction": "override",
  "defaultOverrideAction": "leave",
  "parameters": [
    {
      "name": "@ServiceAttribute",
      "type": "ServiceAttribute",
      "required": "true",
      "description": "Select the service attribute for the service filter"
    }
  ],
  "dependencies": [
    {
      "qappId": "6a376576-58b0-4c67-8df9-e0c0516f4aeb",
      "qappName": "Customer Data Erasure",
      "minimumVersion": "",
      "downloadUrl": "https://qmarket.qnomy.com/market/details/6a376576-58b0-4c67-8df9-e0c0516f4ae
    }
```

```
],
  "content": [
    {
      "storage": {
        "name": "Storage Files",
        "type": "folder",
        "path": "Content",
        "target": "SampleKioskv6Content",
        "rollbackAction": "remove",
        "upgradeAction": "override"
      }
    },
    {
      "import": {
        "name": "Kiosk Demo",
        "type": "ReceptionPointProfile",
        "path": "Import/ReceptionPointProfile_Demo.xml"
      }
    },
    {

      "dbScript": {
        "name": "Create Stored Procedure",
        "type": "storedProcedure",
        "path": "Scripts/sp.sql",
        "target": "qflow",
        "rollbackPath": "Scripts/rollbackSp.sql"
      }
    },
    {
      "manual": {
        "name": "Script Dll",
        "path": "serviceScript.dll",
        "instructions": "Download and copy the file to the BIN folder on every Q-Flow web server",
        "rollbackInstructions": "Remove the file 'serviceScript.dll' from the BIN folder"
      }
    }
  ]
```

Zip & go

The `qapp` folder containing all the package's components needs to be archived as a zip file with the Q-App's name (noting the version is useful). Q-Flow identifies and processes only archived files with the `.qapp` extension as Q-Apps. Therefore after zipping the file you need to change the file's extension manually from ".zip" to ".qapp".

See also

- Creating Folder Hierarchies in Q-Apps
- Q-App Compression Limitations
- Q-Flow Apps
- Uploading and Installing Q-Apps
- How to use Q-App Parameters
- Installing Q-Apps Comprising SQL DB Scripts
- Q-Flow Apps Manager
- Q-App Submission to Q-Market Guidelines